

Lec 7

أسئلة

1. What does a Seq2Seq model consist of?

- A) Only an encoder
- B) Only a decoder
- C) An encoder and a decoder ☒
- D) Only attention

Explanation: Seq2Seq models have an **encoder** that processes input and a **decoder** that generates output.

2. What is the main role of the encoder?

- A) Generate output sequence
- B) Encode the input sequence into a context vector ☒
- C) Apply attention
- D) Compute softmax

Explanation: Encoder transforms input sequence into a **fixed-length vector** or sequence of hidden states.

3. What problem occurs when encoding long sequences into a fixed vector?

- A) Overfitting
- B) Underfitting
- C) Bottleneck / vanishing gradient ☒
- D) Normalization issues

Explanation: Compressing long sequences into a single vector can **lose information**.

4. In Seq2Seq, the decoder generates the output sequence

- A) Randomly
- B) Based only on previous words
- C) Based on previous words and context vector ☒
- D) Using only the last hidden state

Explanation: Decoder uses **previous generated words + context vector** to predict the next word.

5. Which neural network is commonly used in Seq2Seq encoders?

- A) CNN
- B) RNN / LSTM ☒
- C) Transformer only
- D) GAN

Explanation: RNNs and LSTMs are popular for sequence modeling.

6. What does the context vector represent?

- A) Next word
- B) Entire input information ☒
- C) Decoder state
- D) Attention weight

Explanation: Context vector is a **summary of the input sequence**.

7. Why was attention introduced?

- A) To make Seq2Seq faster
- B) To handle long sequences better ☒
- C) To replace the decoder
- D) To normalize hidden states

Explanation: Attention allows the model to **focus on relevant parts of input** instead of one fixed vector.

8. In attention, what does α_{ij} represent?

- A) Hidden state
- B) Input word
- C) Importance of input word j for output word i ☒
- D) Output word probability

Explanation: α_{ij} = weight of how much the j -th input word matters for generating i -th output word.

9. What is self-attention?

- A) Attention to previous outputs
- B) Attention to a sequence itself ☒
- C) Attention to context vector only
- D) Attention to the decoder state

Explanation: Self-attention relates **parts of the sequence with each other** to build representations.

10. Which task can use self-attention?

- A) Sentiment analysis ☒
- B) Image compression
- C) Object detection
- D) GAN training

Explanation: Self-attention is useful for **NLP tasks**, e.g., understanding relationships between words.

11. In image captioning, attention is used to:

- A) Compress image
- B) Focus on relevant image regions ☒
- C) Generate random pixels
- D) Replace CNN

Explanation: Attention highlights **important regions of the image** when generating each word.

12. Transformers do NOT use:

- A) Attention
- B) RNNs / LSTMs ☒
- C) Queries, Keys, Values
- D) Multi-head attention

Explanation: Transformers rely **entirely on attention**, not recurrence.

13. Scaled Dot-Product Attention formula includes:

- A) $Q + K + V$
- B) $\text{Softmax}(QK^T / \sqrt{d_k}) V$ ☒
- C) QK^T / V
- D) $Q * K * V$

Explanation: Scaling by $\sqrt{d_k}$ **prevents large values** that can reduce gradient efficiency.

14. What are Queries, Keys, Values?

- A) Types of loss functions
- B) Representations in attention mechanism ☒
- C) CNN filters
- D) Activation functions

Explanation: Q,K,V are **vectors used to compute attention weights**.

15. What is the purpose of multi-head attention?

- A) Reduce parameters
- B) Increase model capacity by attending to multiple positions ☒
- C) Replace decoder
- D) Only for vision tasks

Explanation: Multiple heads allow **simultaneous attention to different parts of sequence**.

16. In multi-head self-attention, Q, K, V are:

- A) Random vectors
- B) Output embeddings
- C) Input representations from the same layer ☒
- D) Only context vector

Explanation: Self-attention uses **input representations as $Q=K=V$** .

17. Why scale dot products by $\sqrt{d_k}$?

- A) To increase training speed
- B) To prevent softmax gradients from being too small ☒
- C) To reduce model size
- D) To normalize outputs

Explanation: Large dot products can **push softmax to regions with tiny gradients**.

18. What does the decoder in attention models decide?

- A) Learning rate
- B) Which input words to focus on ☒
- C) Hidden state initialization
- D) Sequence length

Explanation: Decoder chooses **relevant input positions** for generating each word.

19. Alignment model in attention is usually:

- A) Feedforward neural net ☒
- B) CNN

- C) LSTM
- D) RNN only

Explanation: Alignment scores e_{ij} are computed using a **small feedforward network**.

20. Which problem does attention solve in Seq2Seq?

- A) Underfitting
- B) Computation cost
- C) Information bottleneck ☒
- D) Embedding size

Explanation: Attention **avoids squeezing all input info** into one vector.

21. In transformers, self-attention can capture:

- A) Local word relationships only
- B) Long-range dependencies ☒
- C) Only first word
- D) Only last word

Explanation: Self-attention captures **relationships between any words** in the sequence.

22. Which of these tasks has used attention successfully?

- A) Machine translation ☒
- B) Sorting arrays
- C) Sorting numbers
- D) Matrix inversion

Explanation: NMT (Neural Machine Translation) is **classic attention application**.

23. The probability of generating a word in decoder is conditioned on:

- A) Context vector only
- B) Previous words only
- C) Context vector and previous words ☒
- D) Encoder weights only

Explanation: Each word depends on **both previous outputs and input info**.

24. In Seq2Seq with attention, context vector is:

- A) Fixed for all output words
- B) Varies per output word ☒
- C) Not used
- D) Always zero

Explanation: Attention produces **dynamic context vectors** for each output word.

25. Which paper introduced Transformers?

- A) Show, Attend and Tell
- B) Attention Is All You Need ☒
- C) LSTM Networks for Machine Reading
- D) Seq2Seq with Attention

Explanation: Vaswani et al., 2017 introduced **the Transformer architecture**.

26. One advantage of self-attention over RNNs is:

- A) Slower computation
- B) Parallel computation ☒
- C) Cannot capture long sequences
- D) Requires CNN

Explanation: Self-attention allows **parallel processing of sequences**, unlike RNNs.

27. Multi-head attention concatenates heads then:

- A) Adds them
- B) Projects them linearly ☒
- C) Ignores some heads
- D) Normalizes softmax

Explanation: After concatenation, a **linear projection** produces final values.

28. Self-attention is also called:

- A) Intra-attention ☒
- B) Cross-attention
- C) Temporal attention
- D) Feedforward attention

Explanation: It relates **parts of a single sequence**, hence “intra-attention.”

29. Attention mechanism in vision helps:

- A) Compress hidden states
- B) Focus on important image regions ☒
- C) Replace Transformers
- D) Increase context vector size

Explanation: Helps generate **relevant words for regions in the image**.

30. Key breakthrough of attention in NLP is similar to:

- A) GANs in vision
- B) CNNs in vision ☒
- C) RNNs in vision
- D) PCA in vision

Explanation: Attention **revolutionized NLP like CNNs did for vision**.

Transformer $\rightarrow$ BERT:

1. How many identical layers does a standard Transformer encoder have?

- A) 4
- B) 6 ☒
- C) 8
- D) 12

Explanation: The original Transformer encoder has **6 layers**.

2. What are the two sub-layers in a Transformer encoder layer?

- A) Self-attention and RNN
- B) Multi-head attention and feed-forward network ☒
- C) CNN and attention
- D) Embedding and softmax

Explanation: Each encoder layer has **multi-head attention + feed-forward network**.

3. What is used in each sub-layer to help gradient flow?

- A) Dropout
- B) Residual connection + layer normalization ☒

- C) ReLU
- D) Batch normalization

Explanation: **Residual connections + layer norm** prevent vanishing gradients.

4. What extra sub-layer does the decoder have?

- A) Feed-forward network
- B) Masked attention
- C) Multi-head attention over encoder outputs ☒
- D) Embedding layer

Explanation: The decoder attends to encoder outputs via an **extra attention sub-layer**.

5. What does masked self-attention do in the decoder?

- A) Attends to future words
- B) Prevents attending to subsequent positions ☒
- C) Normalizes hidden states
- D) Compresses input

Explanation: Ensures each output depends **only on previous outputs**.

6. What is the purpose of positional encoding?

- A) Prevent overfitting
- B) Indicate the order of words in sequence ☒
- C) Scale embeddings
- D) Add attention weights

Explanation: Attention is **permutation-invariant**, so we add positional info.

7. What type of functions did the original Transformer use for positional encoding?

- A) Sigmoid & tanh
- B) Sine & cosine ☒
- C) ReLU & Leaky ReLU
- D) Softmax & exp

Explanation: Original paper uses **sin/cos of different frequencies**.

8. Transformers can process sequences:

- A) Sequentially
- B) In parallel ☒
- C) Only one token at a time
- D) Randomly

Explanation: Transformers work **in parallel**, unlike RNNs.

9. What is a main advantage of Transformers over RNNs?

- A) Less memory
- B) Long-term dependency handling ☒
- C) Slower computation
- D) Smaller datasets

Explanation: Self-attention captures **long-range dependencies easily**.

10. What is a disadvantage of Transformers?

- A) Cannot handle sequences
- B) Quadratic scaling with input length ☒
- C) Cannot use attention
- D) Sequential computation only

Explanation: Attention scales **$O(n^2)$** with sequence length.

11. BERT stands for:

- A) Bidirectional Encoder Representations from Transformers ☒
- B) Binary Encoding Representation Transformer
- C) Backward Encoder for RNN Transformers
- D) Bidirectional Encoding Recursive Transformer

Explanation: BERT = **Bidirectional Encoder Representations from Transformers**.

12. What makes BERT bidirectional?

- A) It looks only forward
- B) It conditions on both left and right context ☒
- C) It uses RNN
- D) It uses CNN

Explanation: BERT learns **context from both sides** of a token.

13. How can BERT be fine-tuned for tasks?

- A) Add multiple RNN layers
- B) Add a single output layer ☒
- C) Retrain entire network only
- D) No fine-tuning possible

Explanation: Pre-trained BERT can be fine-tuned with **one task-specific layer**.

14. What is the first token in every BERT input sequence?

- A) [SEP]
- B) [CLS] ☒
- C) [MASK]
- D) [PAD]

Explanation: [CLS] token represents **aggregate sequence embedding**.

15. How does BERT handle sentence pairs?

- A) Treat them separately
- B) Pack them into one sequence, separated by [SEP] ☒
- C) Only use the first sentence
- D) Encode only token embeddings

Explanation: Sentence pairs are **combined with [SEP] token**.

16. BERT input embedding =

- A) Token embeddings only
- B) Token + segment + positional embeddings ☒
- C) Only positional embeddings
- D) Segment embeddings only

Explanation: Input = sum of **token, segment, position embeddings**.

17. How many heads are typical in multi-head attention?

- A) 2
- B) 4
- C) 8 ☒
- D) 16

Explanation: Original Transformer uses **8 attention heads**.

18. How many feed-forward units in each layer (dim_feedforward)?

- A) 512
- B) 1024
- C) 2048 ☒
- D) 4096

Explanation: Each feed-forward network typically has **2048 hidden units**.

19. Which operation is attention permutation-invariant?

- A) Masked attention
- B) Multi-head attention ☒
- C) Feed-forward
- D) Embedding

Explanation: Attention doesn't care about **order unless positional encoding is added**.

20. What family of models does GPT belong to?

- A) Vision Transformer
- B) Language Transformer ☒
- C) Multimodal Transformer
- D) CNN

Explanation: GPT is a **language-only Transformer**.

21. CLIP and Flamingo are examples of:

- A) Vision Transformers
- B) Multimodal Transformers ☒
- C) Language-only Transformers
- D) RNN models

Explanation: CLIP/Flamingo combine **text + image** input.

22. What is the role of the residual connection in Transformer layers?

- A) Increase output size
- B) Facilitate gradient flow ☒
- C) Replace feed-forward network
- D) Encode position

Explanation: Residual connection **prevents vanishing gradients**.

23. The encoder output in Transformer is often called:

- A) Memory ☒
- B) Attention
- C) Hidden state
- D) Positional encoding

Explanation: Encoder output = **memory** for decoder attention.

24. What prevents the decoder from “cheating” and looking ahead?

- A) Positional encoding
- B) Masked self-attention ☒
- C) Layer normalization
- D) Feed-forward network

Explanation: Masked attention ensures **future tokens are not seen**.

25. BERT can be used for:

- A) Classification
- B) Question answering
- C) NER (Named Entity Recognition)
- D) All of the above ☒

Explanation: BERT is **general-purpose for NLP tasks**.

26. The output of [CLS] token in BERT is used for:

- A) Token-level prediction
- B) Sequence-level representation ☒
- C) Positional encoding
- D) Multi-head attention

Explanation: [CLS] hidden state = **aggregate sequence embedding**.

27. Which type of attention does BERT use?

- A) Unidirectional
- B) Bidirectional self-attention ☒

- C) Masked attention only
- D) CNN-based attention

Explanation: BERT uses **bidirectional self-attention**.

28. What does the decoder layer attend to in addition to self-attention?

- A) Future tokens
- B) Encoder outputs ☒
- C) Positional encoding
- D) Feed-forward only

Explanation: Decoder attends to **encoder memory** for context.

29. Pros of Transformer compared to RNN:

- A) Sequential computation
- B) Can process in parallel ☒
- C) Cannot handle long dependencies
- D) Requires less memory

Explanation: Transformer allows **parallel computation** and handles **long-term dependencies**.

30. Original Transformer paper:

- A) Vaswani et al., 2017 ☒
- B) Devlin et al., 2018
- C) Xu et al., 2016
- D) Bahdanau et al., 2015

Explanation: “Attention Is All You Need” paper introduced the Transformer.

BERT, GPT, CLIP, Flamingo وTransformers:

1. What does MLM in BERT stand for?

- A) Masked Language Modeling ☒
- B) Multi-layered Machine
- C) Maximum Likelihood Model
- D) Multi-Level Mask

Explanation: MLM = Mask some tokens and predict them.

2. What is the main task of NSP in BERT pretraining?

- A) Predict the next word
- B) Predict if sentence B follows sentence A ☒
- C) Translate sentences
- D) Classify words

Explanation: NSP checks if sentence B is the actual next sentence 50% of the time.

3. Which datasets were used to pre-train BERT?

- A) ImageNet and COCO
- B) BooksCorpus and Wikipedia ☒
- C) CIFAR-10 and MNIST
- D) SQuAD and GLUE

Explanation: BERT trained on text corpora: BooksCorpus (800M words) + Wikipedia (2.5B words).

4. What type of embeddings does BERT produce?

- A) Static embeddings like Word2Vec
- B) Contextualized embeddings ☒
- C) Random embeddings
- D) One-hot vectors

Explanation: Same word gets different embeddings depending on context.

5. Which variant of BERT is optimized for longer training and bigger data?

- A) ALBERT
- B) RoBERTa ☒
- C) DistilBERT
- D) ClinicalBERT

Explanation: RoBERTa uses more data, longer training, and larger batches.

6. What is GPT mainly trained to do?

- A) Predict masked words
- B) Predict the next token in sequence ☒
- C) Translate languages
- D) Classify images

Explanation: GPT is a generative language model (unidirectional decoder).

7. GPT is an example of which type of Transformer?

- A) Bidirectional encoder
- B) Unidirectional decoder ☒
- C) Multimodal encoder
- D) Masked autoencoder

Explanation: GPT predicts tokens left-to-right, one token at a time.

8. What does “auto-regression” mean in GPT?

- A) Model uses future tokens
- B) Model adds generated tokens to input for next prediction ☒
- C) Model trains without supervision
- D) Model predicts sentences randomly

Explanation: Each generated token is fed back as input to predict the next one.

9. Which GPT model has 175 billion parameters?

- A) GPT-1
- B) GPT-2
- C) GPT-3 ☒
- D) GPT-4

Explanation: GPT-3 is massive with 175B parameters.

10. Which BERT feature allows it to understand sentence pairs?

- A) Masked LM
- B) Positional encodings
- C) Segmentation embeddings ☒
- D) Auto-regression

Explanation: Segmentation embeddings indicate sentence A or B.

11. What is the key difference between BERT and GPT?

- A) BERT is generative, GPT is not
- B) GPT is bidirectional, BERT is unidirectional
- C) BERT is bidirectional encoder, GPT is unidirectional decoder ☒
- D) GPT uses masked LM, BERT does not

Explanation: BERT sees both left & right context, GPT only left-to-right.

12. What does CLIP stand for?

- A) Contrastive Language-Image Pretraining ☒
- B) Classification of Language Inputs
- C) Contextual Language-Image Processing
- D) Cross-Lingual Image Pretraining

Explanation: CLIP learns a joint embedding space for text and images.

13. What type of loss does CLIP use?

- A) Cross-entropy
- B) Contrastive loss ☒
- C) Mean squared error
- D) Hinge loss

Explanation: CLIP maximizes similarity for true pairs, minimizes for incorrect pairs.

14. What is the main advantage of CLIP at inference?

- A) Generates captions
- B) Zero-shot classification ☒
- C) Fine-tunes quickly
- D) Produces embeddings randomly

Explanation: CLIP can classify images without extra training.

15. What is Flamingo used for?

- A) Pure NLP tasks
- B) Multimodal few-shot tasks ☒
- C) Image-only classification
- D) Statistical machine translation

Explanation: Flamingo handles text + image in few-shot settings.

16. Which task can Flamingo perform?

- A) Image captioning ☒
- B) Text classification only

- C) Image segmentation only
- D) Predict masked tokens only

Explanation: Flamingo can generate text based on images (open-ended).

17. In BERT, the [CLS] token is used for:

- A) Masked LM
- B) Sequence classification ☒
- C) Image embedding
- D) Auto-regression

Explanation: The [CLS] hidden state represents the whole sequence for classification.

18. What prevents the Transformer decoder from seeing future tokens?

- A) Feedforward network
- B) Masked self-attention ☒
- C) Positional encoding
- D) Residual connection

Explanation: Masked self-attention blocks future positions during decoding.

19. Why do Transformers need positional encodings?

- A) To normalize layers
- B) To know the order of words ☒
- C) To mask tokens
- D) To predict next sentence

Explanation: Attention alone is permutation-invariant, so we add positional info.

20. Which attention variant relates parts of the same sequence?

- A) Multi-head attention
- B) Self-attention ☒
- C) Cross-attention
- D) Scaled-dot product attention

Explanation: Self-attention compares each token with others in the same sequence.

21. Which Transformer component increases model capacity by using multiple attention projections?

- A) Layer normalization
- B) Multi-head attention ☒
- C) Positional encoding
- D) Feedforward network

Explanation: Each head learns different aspects of the sequence.

22. What is the main limitation of traditional seq2seq models?

- A) Cannot handle sequences at all
- B) Fixed-length context vector ☒
- C) Too fast to train
- D) No embeddings

Explanation: Long sequences are hard to compress into a single context vector.

23. What breakthrough solved seq2seq bottleneck issues?

- A) Transformers
- B) Attention mechanism ☒
- C) Positional encodings
- D) RNN

Explanation: Attention lets the decoder focus on relevant input tokens.

24. What is the core idea of scaled dot-product attention?

- A) Scale input embeddings
- B) Scale dot product by $\sqrt{d_k}$ ☒
- C) Scale output embeddings
- D) Normalize batch size

Explanation: Prevents large dot products from causing tiny gradients.

25. Which models are “contextualized word embeddings” models?

- A) Word2Vec
- B) GloVe
- C) BERT ☒
- D) One-hot

Explanation: Contextual embeddings depend on surrounding words.

26. GPT models perform in-context learning. What does it mean?

- A) Learning without seeing input
- B) Conditioned on instructions or examples ☒
- C) Only predicting masked words
- D) Fine-tuning on large dataset

Explanation: GPT can adapt to tasks using few examples in the prompt.

27. Flamingo differs from CLIP in that it can:

- A) Only classify images
- B) Generate open-ended text from images ☒
- C) Only embed text
- D) Only do zero-shot image retrieval

Explanation: Flamingo can generate text, not just compute similarity.

28. Which BERT variant is designed to be lighter for efficiency?

- A) RoBERTa
- B) ALBERT ☒
- C) GPT-2
- D) CLIP

Explanation: ALBERT reduces parameters while keeping performance.

29. What is the function of residual connections in Transformer layers?

- A) Mask future tokens
- B) Add input to sub-layer output to stabilize training ☒
- C) Generate embeddings
- D) Compute attention scores

Explanation: Helps gradient flow and stabilizes deep networks.

30. Which of the following is true about Transformers vs RNNs?

- A) Transformers are sequential, slower
- B) Transformers process in parallel, handle long-term dependencies ☒
- C) RNNs scale better to long sequences
- D) Transformers cannot do NLP

Explanation: Transformers are parallelizable and can handle long sequences efficiently.

Flamingo, ViT, Perceiver, Transformers

1–10: Transformers & Vision

1. What is the main advantage of using attention over convolutions?

- A) Requires less data
- B) Captures long-range dependencies
- C) Faster for small images
- D) Reduces overfitting

✅ **Answer: B**

Explanation: Attention can connect distant parts of input, unlike CNNs which focus on local neighborhoods.

2. In Vision Transformer (ViT), an image is split into:

- A) Pixels
- B) Channels
- C) Patches
- D) Filters

✅ **Answer: C**

Explanation: ViT treats each patch like a “word” in NLP.

3. What is the [CLS] token in ViT used for?

- A) Image reconstruction
- B) Classification
- C) Patch embedding
- D) Data augmentation

✅ **Answer: B**

Explanation: Like BERT, the [CLS] token summarizes the sequence for classification.

4. Why are positional encodings added in ViT?

- A) To normalize patches
- B) To retain spatial order
- C) To reduce model size
- D) To increase channels

✅ **Answer: B**

Explanation: Attention alone is permutation-invariant, so we add positions to know patch order.

5. ViT can be trained without CNNs because:

- A) Patches replace convolution
- B) Attention handles spatial relationships
- C) Positional encodings encode shapes
- D) All of the above

✅ **Answer: D**

Explanation: Attention + patches + positions allow ViT to process images directly.

6. What is the computational complexity of self-attention on an image of n patches?

- A) $O(n)$
- B) $O(n^2)$
- C) $O(\log n)$
- D) $O(1)$

✓ **Answer: B**

Explanation: Each patch attends to all other patches $\rightarrow$ quadratic.

7. Swin Transformer improves efficiency by:

- A) Using CNNs
- B) Adding hierarchy and local attention
- C) Using fewer layers
- D) Removing positional encodings

✓ **Answer: B**

Explanation: Local attention reduces computation from $O(n^2)$ to smaller regions.

8. The original ViT needed ImageNet-22k because:

- A) Model is too small
- B) Transformers require large data
- C) CNNs are used inside
- D) Positional encoding requires it

✓ **Answer: B**

Explanation: Transformers have no convolutional inductive bias $\rightarrow$ need lots of data.

9. DeiT paper showed ViT can be trained on:

- A) ImageNet-1k with more augmentations
- B) Only synthetic images
- C) MNIST only
- D) CIFAR-10 without augmentations

✓ **Answer: A**

Explanation: Data-efficient training + augmentations enable smaller dataset training.

10. Feature maps in ViT are:

- A) Variable size
- B) Constant size
- C) Only one channel
- D) Not used

✓ **Answer: B**

Explanation: Constant feature map size makes downstream tasks easier.

11–20: Flamingo & Multimodal Transformers

11. Flamingo model combines:

- A) Only text
- B) Only image
- C) Image + text

D) Audio + text

✓ **Answer: C**

Explanation: Flamingo is multimodal for visual-language tasks.

12. On the vision side, Flamingo uses:

A) CNN

B) Vision encoder (CLIP-style)

C) ViT only

D) Autoencoder

✓ **Answer: B**

Explanation: CLIP-style encoder produces embeddings from images.

13. On the language side, Flamingo uses:

A) Transformer encoder

B) Autoregressive LM

C) CNN

D) RNN

✓ **Answer: B**

Explanation: Pretrained LM generates text conditioned on visual tokens.

14. Flamingo links vision and language via:

A) Convolution

B) Learnable attention (Perceiver)

C) Recurrent layers

D) Pooling layers

✓ **Answer: B**

Explanation: Attention module maps visual tokens to LM input.

15. Flamingo can do few-shot learning because:

A) It has pre-trained weights

B) It uses in-context learning like GPT

C) It ignores images

D) It uses CNNs

✓ **Answer: B**

Explanation: Flamingo learns from a few examples by conditioning on prompts.

16. Flamingo outputs:

A) Fixed-size visual tokens

B) Variable text length only

C) Image reconstruction

D) Classification labels only

✓ **Answer: A**

Explanation: Visual embeddings condition the LM for text generation.

17. Zero-shot performance in Flamingo means:

A) Model trained on task data

B) Model predicts without seeing labeled examples

- C) Model cannot generalize
- D) Model uses CNNs

✅ **Answer: B**

Explanation: Model uses embeddings and pretraining to predict new tasks without training.

18. CLIP uses which loss function?

- A) Cross-entropy
- B) MSE
- C) Contrastive loss
- D) L1 loss

✅ **Answer: C**

Explanation: Maximizes cosine similarity for correct image-text pairs, minimizes for wrong pairs.

19. At inference, CLIP predicts:

- A) Image reconstruction
- B) Most probable image-text pair
- C) Only text generation
- D) Only image classification

✅ **Answer: B**

Explanation: Uses cosine similarity + softmax to match images with captions.

20. CLIP's limitation is:

- A) Cannot classify
- B) Cannot generate open-ended text
- C) Cannot process images
- D) Requires few data

✅ **Answer: B**

Explanation: CLIP only outputs similarity scores; it's not a generative model.

21–30: Perceiver & General Transformers

21. Perceiver solves:

- A) Only NLP tasks
- B) Scaling attention to large, multimodal inputs
- C) Only image tasks
- D) Only audio tasks

✅ **Answer: B**

Explanation: Perceiver projects large inputs to a smaller latent space.

22. Perceiver reduces complexity from $O(M^2)$ to:

- A) $O(M)$
- B) $O(MN)$
- C) $O(N^2)$
- D) $O(1)$

✅ **Answer: B**

Explanation: Latent array of size $N \ll M$ reduces attention cost.

23. In Perceiver, Q comes from:

- A) Input data
- B) Learned latent array
- C) Output embeddings
- D) Positional encoding

✅ **Answer: B**

Explanation: Latent array is smaller and learned to attend efficiently.

24. In Perceiver, K and V come from:

- A) Input data
- B) Latent array
- C) Positional embeddings
- D) Output sequence

✅ **Answer: A**

Explanation: Original input provides key and value projections.

25. Transformers achieve SOTA because:

- A) Pre-training on large data + fine-tuning
- B) Use of CNNs
- C) Small datasets
- D) Simple RNN layers

✅ **Answer: A**

Explanation: Large self-supervised pretraining gives transferable features.

26. BERT is:

- A) Autoregressive LM
- B) Bidirectional Transformer encoder
- C) Vision model only
- D) Unimodal audio model

✅ **Answer: B**

Explanation: BERT reads left and right context simultaneously.

27. GPT is:

- A) Bidirectional encoder
- B) Unidirectional decoder
- C) Vision-only
- D) Perceiver variant

✅ **Answer: B**

Explanation: GPT predicts next token autoregressively.

28. GPT's auto-regression means:

- A) Uses CNNs for each token
- B) Adds each predicted token to input for next step
- C) Ignores previous tokens
- D) Uses contrastive learning

✅ **Answer: B**

Explanation: Sequence grows with each predicted token for context.

29. Flamingo can do:

- A) Image captioning
- B) Visual question answering
- C) Visual dialogue
- D) All of the above

✅ Answer: D

Explanation: Multimodal attention allows many visual-language tasks.

30. The general recipe for Transformers' success is:

- A) Small models + supervised learning
- B) Self-supervised pretraining + fine-tuning
- C) Only CNNs + RNNs
- D) Random initialization + few-shot

✅ Answer: B

Explanation: Pretrain on large data, fine-tune on task-specific small data → SOTA performance.

ملخص

Seq2seq Models

- **Encoder-Decoder** دي موديلات
- شغالة جامد في ترجمة النصوص، تلخيص النصوص، وتوصيف الصور
- يولد الترجمة بناءً على الـ **decoder** ، وبعدين الـ (context vector) واحد vector يقرأ الجملة ويمثلها في الـ **encoder** الـ الفكرة ده context

Encoder

- hidden states بياخد جملة المصدر ويحولها لسلسلة
- من السلسلة دي **context vector** بعد كده بيطلع
- عشان يقدر يتعامل مع التسلسل **LSTM** أو **RNN** غالبًا بنستخدم
- $h_t = f(x_t, h_{t-1})$
- $c = q(\{h_1, h_2, \dots, h_T\})$
- **h_t**: الحالة المخفية في وقت t
- **c**: الـ context vector
- **f, q**: دوال غير خطية

Decoder

- مدرب عشان يتنبأ بالكلمة الجاية بناءً على:
 1. context vector الـ
 2. الكلمات اللي اتولدت قبل كده
- يحسب احتمالية الجملة بالتتابع:
- $p(y) = \prod p(y_t | y_1, \dots, y_{(t-1)}, c)$
- بـ decoder modeled كل خطوة في الـ
- $p(y_t | \dots) = g(y_{(t-1)}, s_t, c)$
- دالة غير خطية: **g**
- RNN الحالة المخفية للـ: **s_t**

التقليدي Seq2seq مشكلة

- **bottleneck** ده بيبقى → ثابت الحجم **vector** كل المعلومات لازم تتحط في
- **vanishing gradient** ممكن ينسى حاجات مهمة بسبب مشكلة RNN خصوصًا لو الجمل طويلة → الـ

Attention

- bottleneck حل مشكلة الـ
- واحد، بنخلي الموديل يركز على أجزاء معينة من الجملة المصدرية كل مرة بيولد كلمة vector بدل ما نضغط كل المعلومات في
- encoder للـ hidden states خاص بيتم حسابه من كل **context vector** كل كلمة هدف ليها
- بيبين أهمية كل كلمة مصدرية بالنسبة للكلمة الهدف α_{ij} الوزن

Self-Attention

- بيقارن أجزاء التسلسل مع بعضه attention نوع من
- بيطلع تمثيل للجملة كلها
- زي تحليل المشاعر NLP مفيد في

Attention (Vision) في الرؤية

- الموديل بيتعلم يركز على مناطق الصورة المهمة أثناء توليد الكلمات: **Image Captioning** زي في

Transformer

- CNN ولا RNN ، مفيهوش **attention** كله Encoder-Decoder موديل
- ، غير شكل الموديلات خالص NLP في ImageNet شبه الـ
- أفكار مهمة: **Queries, Keys, Values**
- **Scaled Dot-Product Attention:**
- $\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$
 - تمثيلات للكلمات: Q, K, V
 - softmax عشان ما تكبرش القيم وتخرّب الـ $\sqrt{d_k}$

Multi-Head Attention

- Q, K, V مختلفة من projections أكثر من مرة على **attention** بنعمل
- بندمج النتائج عشان الموديل يقدر يركز على حاجات مختلفة في نفس الوقت
- Multi-head self-attention: $Q=K=V$ = نفس layer تمثيلات من نفس previous layer.

Self-Attention مميزات

- ببسّهل تفسير الموديل → ممكن تشوف الموديل بيركز على إيه في كل خطوة
- بعيدة في الجملة **dependencies** ببسّاعد الموديلات على فهم

1. Transformer Encoder

- (N=6) متشابهة **6 layers** فيه
- **2 sub-layers** فيها كل layer
 1. **Multi-head attention**
 2. **Feed-forward network**
- **residual connection + layer normalization** عليه كل sub-layer **vanishing** ده بيخلي التعلم أسهل وما يحصلش **gradient**.

2. Transformer Decoder

- متشابهة **6 layers** برضو فيه
- ثالث إضافي **layer** بس فيه **encoder** زي الـ layer كل
 - **encoder** بتاع **output** على **Multi-head attention**

- عشان كل كلمة تتولد تعتمد على الكلمات اللي قبلها بس وما تبصش لقدام **masked self-attention** فيه
-

3. Transformer Code (PyTorch)

- (مثلاً 512) embedding عدد الخصائص لكل d_{model} عندنا
 - (مثلاً 8) multi-head attention عدد رؤوس الـ n_{head}
 - (عادة 6) layers عدد $num_encoder_layers, num_decoder_layers$
 - (مثلاً 2048) feed-forward في hidden layer حجم $dim_feedforward$
 - $forward(src, tgt)$ output من encoder على الـ decoder وبعدين src على الـ encoder أولاً
-

4. Positional Encodings

- لو قلبت الكلمات هتجيب نفس النتيجة → لا تفرق ترتيب الكلمات **attention**: المشكلة
 - عشان الموديل يعرف ترتيب الكلمات input لكل **positional encoding** الحل: نضيف
 - بتترددات مختلفة **sine & cosine**: الطريقة الأصلية
 - الهدف: الموديل يعرف المسافة بين الكلمات وموضعها في التسلسل
-

5. Pros & Cons of Transformer

Pros:

- RNN أسرع من $\rightarrow$ **parallel** بيشتغل
- بسهولة **long-term dependencies** بيتعامل مع

Cons:

- مع طول التسلسل **quadratically** بيكبر
 - محتاج ذاكرة كبيرة وبيانات كتير ووقت تدريب طويل
-

6. Summary of Architecture

- deep learning لأي مشكلة Encoder-decoder useful
 - صعب يضغط كل المعلومات **context vector**: NMT: المشكلة التقليدية في
 - حل المشكلة دي $\rightarrow$ بيختار أهم المعلومات Attention
 - بس **attention** كله بيعتمد على Transformer
-

7. Recommended Papers

- NMT with Attention: Bahdanau et al., 2015
 - LSTM for Machine Reading: Cheng et al., 2016
 - Image Captioning with Attention: Xu et al., 2016
 - Transformer: Vaswani et al., 2017
-

8. Transformer Family

- Language Transformers: **BERT, GPT**
 - Multimodal Transformers: **CLIP, Flamingo**
 - Vision Transformers: **ViT, Perceiver**
-

9. BERT

- BERT = **Bidirectional Encoder Representations from Transformers**
 - Pre-trained على نصوص كبيرة بدون labels
 - (bidirectional) يعرف السياق من اليمين والشمال
 - NLP → SOTA performance مع طبقة واحدة لأي مهمة **fine-tune** ممكن
-

10. BERT Input Representation

- ممكن يمثل جملة واحدة أو جملتين
 - Input = **token embeddings + segment embeddings + positional embeddings**
 - sequence/الجملة representative يتأخره هو [CLS] → final hidden state دائماً token أول
 - B ولا A بيقول الجملة embedding ، وكمان فيه [SEP] لو جملتين: يفصلهم
-

بطريقة مختصرة وبالعامة عشان تفهمه بسرعة **BERT, GPT, CLIP و Flamingo** تمام! هاشرك اللى انت بعته عن

1. BERT Pre-training

- حقيقة NLP عنده تدريب مسبق قبل ما يستخدم في مهام
- فيه نوعين من المهام:

1. Masked Language Modeling (MLM):

- [MASK] بنحفي شوية كلمات عشوائي في الجملة بـ
- النموذج يحاول يتوقع الكلمات المخفية

2. Next Sentence Prediction (NSP):

- صح ولا لا A هي الجملة اللي جاية بعد B النموذج يقرر هل الجملة
 - الجملة الصحيحة، 50% جملة عشوائية 50%
 - (مليار كلمة 2.5) Wikipedia (800 مليون كلمة) و BooksCorpus: البيانات اللي اتدرب عليها
-

2. BERT Fine-Tuning

- على مهام محددة بسهولة BERT بعد التدريب المسبق، تقدر تعدل
 - سواء المهمة جملة واحدة أو جملتين
 - رخيص نسبيًا مقارنة بالتدريب المسبق
-

3. BERT vs Word2Vec/GloVe

- Word2Vec/GloVe بيعملوا تمثيل ثابت للكلمات، يعني كلمة واحدة نفس التمثيل في أي جملة
 - BERT contextualized embeddings بيعمل:
 - هتكون مختلفة لو الجملة عن نهر أو بنك "bank" مثال: كلمة
-

4. Variants of BERT (BERTology)

- في نسخ محسنة أو أخف:
 - RoBERTa: أكبر batch تدريب أطول، بيانات أكبر،
 - ALBERT: BERT نسخة أخف من
 - DeBERTa, DistilBERT, CamemBERT, ClinicalBERT... إلخ
-

5. GPT (Generative Pretrained Transformer)

- GPT1: Transformer decoder وحيد الاتجاه (left-to-right)
- فكرة GPT:
 1. Pre-train (predict next token) على بيانات غير معنونة
 2. Fine-tune على مهمة محددة

- BERT: الفرق عن
 - BERT: **bidirectional encoder** → يشوف السياق على اليمين والشمال
 - GPT: **unidirectional decoder** → بيولد كلمة كلمة بالترتيب
- GPT (auto-regression) بعد كل كلمة بتولد، بيضيفها على التسلسل ويكمل

6. GPT-1, GPT-2, GPT-3

Model Parameters Decoder Layers Hidden Layer Batch Size

GPT-1	117M	12	768	64
GPT-2	1.5B	48	1600	512
GPT-3	175B	96	12288	3.2M

- كل نسخة أكبر من اللي قبلها، وأقوى في توليد اللغة وفهم السياق
- كود، فقرات طويلة بدون تدريب مباشر على المهمة, SQL ممكن يولد GPT-3

7. In-Context Learning in GPT

- النموذج بياخد تعليمات أو أمثلة قليلة على المهمة
- **predict next token** ويعيد يولد النتائج مباشرة
- language بس بالـ few-shot learning شبه الـ

8. CLIP (Contrastive Language-Image Pretraining)

- **Multimodal Transformer:** نص + صورة
- **Text Encoder** و **Image Encoder** عنده
- التدريب:
 - embeddings قريبين (maximize cosine similarity) → الصورة الصحيحة والنص الصحيح
 - embeddings بعيدين (minimize cosine similarity) → صورة + نص خاطئ
- يتعرف على أصناف جديدة بدون تدريب إضافي → **zero-shot classification**: ميزة
- للتوافق بين صورة ونص score ببديك

9. Flamingo (Visual Language Model)

- (صورة + نص) متعدد الأنماط Transformer
- Few-shot learning: يتعلم من أمثلة قليلة فقط
- عن الصور open-ended يقدر يولد نصوص
- in-context learning يس مع الصور، ويعمل GPT يعني شبه

الخلاصة

- BERT: NLP tasks ، قوي في bidirectional فهم النصوص
- GPT: autoregressive ، unidirectional توليد النصوص
- CLIP: zero-shot classification يربط صورة مع نص
- Flamingo: توليد نصوص + فهم الصور مع أمثلة قليلة

تمام، هاشرك كل النقاط دي بالعامية وبشكل ملخص وسهل للفهم

1. Flamingo: Visual Language Model

- Flamingo (multimodal) عبارة عن نموذج يشغل على نصوص وصور مع بعض
- (embeddings) يحول الصور لتمثيلات رقمية CLIP زي vision encoder فيه: على الجانب البصري
- بيتعلم من نصوص كبيرة (GPT-type) جاهز language model فيه: على جانب اللغة
- (Perceiver زي) learnable attention الإثنين متوصلين ببعض عن طريق
- الفكرة: النموذج بيطلع مجموعة ثابتة من التوكنات البصرية اللي بيستخدمها علشان يولد نصوص متعلقة بالصورة

ممك تسأله على صورة: "إيه اللي حاصل هنا؟" وهو يرد بجمل: مثال عملي

2. Vision Transformer (ViT)

- ، دلوقتي ممكن نستخدمه على الصور مباشرة NLP مش بس لـ Transformer
 - "word" بيتعامل كأنه patch صغيرة (زي 16×16) كل patches ، الصورة مقسمة لـ CNN بدل ما نستخدم
 - على الصورة classification بيتحط في أول التسلسل علشان نعمل BERT زي token [CLS] فيه
 - علشان النموذج يعرف ترتيبهم في الصورة patch لكل positional encodings بنضيف
 - CNNs ممكن يتعلم الصور بدون أي ViT
-

3. Attention vs Convolutions

- مع بعض، ييشوف العلاقة بينهم مباشرة tokens شغال على كل الـ Attention.
 - صغير بس neighborhood بياخد CNN.
 - Advantage: attention ممكن يشوف **long-range dependencies** في الصورة أو النص.
-

4. Training ViT

- علشان يطلع أداء ممتاز (**ImageNet-22k (14M images)** أصلي كان محتاج ViT).
 - صحت augmentations و regularization لو استخدمنا **ImageNet-1k** ورينا ممكن ندرّبه على **DeiT** بعدين.
-

5. ViT Features

- Feature maps سهل computation ثابتة الحجم → سهل downstream tasks.
 - على بيانات كبيرة pretraining زي (SSL) **self-supervised learning** مناسب مع.
-

6. Swin Transformer

- Idea: **local neighborhoods**، نركز على (computational cost) $O(n^2)$ على كل الصورة attention بدل ما نعمل.
 - ده بيقلل الحسابات ويحسن الأداء.
-

7. Hybrid Architectures (LeViT)

- **Transformer + ConvNet** بيجمع بين.
 - أعلى inference الهدف: أداء ممتاز وسرعة.
-

8. The Perceiver

- Challenge: (images, audio, video, point clouds) على مدخلات كبيرة جدًا ومتنوعة attention إزاي نعمل.
 - الفكرة: **latent bottleneck**
 - أصغر latent array من Q للـ project، يعني نعمل $M \ll N$ عندنا.
 - من المدخلات الأصلية الكبيرة V و K.
 - أسرع بكثير → $O(M^2)$ بدل $O(MN)$ بيبقى Complexity.
 - modality بيسمح للتعامل مع أي نوع من البيانات بدون تصميم خاص لكل.
-

9. Summary

- Transformer أصبح أشهر Architecture في **Deep Learning**.
 - Used for:
 - Language: BERT, GPT
 - Multimodal: CLIP, Flamingo
 - Vision: ViT
 - General: Perceiver
 - بسيط لكل مهمة **fine-tuning** + **(self-supervised)** على بيانات ضخمة **pre-training**: الفكرة الناجحة
-